CENTRO UNIVERSITÁRIO UNIBALSAS
Análise e Desenvolvimento de Sistemas

Algoritmos e Programação Avançada
Professor: Alex

RELATÓRIO TÉCNICO – TED 03

Gerenciador de arquivos CSV e JSON com tratamento de exceções e backup automático

Everton Felipe Cirqueira da Silva

Balsas – MA
Outubro de 2026

Sumário

1 Identificação **3**

2 Introdução e objetivos **3**

3 Descrição da solução **4**

3.1 Organização do projeto 4

3.2 Funcionalidades e persistência dos dados 4

4 Implementação **5**

4.1 Gerenciador de contexto e codificação 5

4.2 Leitura de JSON 5

4.3 Escrita e conversão entre CSV e JSON 6

4.4 Tratamento de exceções 7

4.5 Backup automático 7

5 Testes e evidências de execução **8**

5.1 Testes automatizados 8

5.2 Leitura de CSV e conversão para JSON 9

5.3 Leitura de JSON e conversão para CSV 10

5.4 Atualização com geração prévia de backup 11

5.5 Tratamento de erro de arquivo 12

6 Conclusão **12**

7 Referências **13**

1 Identificação

Item	Informação
Instituição	Centro Universitário UNIBALSAS
Curso	Análise e Desenvolvimento de Sistemas
Disciplina	Algoritmos e Programação Avançada
Professor	Alex
Atividade	TED 03 – Gerenciador de arquivos CSV e JSON com tratamento de exceções e backup automático
Integrante(s)	Everton Felipe Cirqueira da Silva
Repositório	gerenciador-csv-json (Git/GitHub)

2 Introdução e objetivos

Aplicações reais precisam guardar informações de forma que elas sobrevivam ao encerramento do programa. Os formatos CSV (valores separados por vírgula) e JSON (*JavaScript Object Notation*) estão entre os mais usados para essa persistência, o primeiro por sua simplicidade tabular e o segundo por representar estruturas de forma hierárquica e legível. Este relatório apresenta um módulo em Python, acompanhado de um menu de terminal, que lê, converte, atualiza e grava dados nesses dois formatos, com atenção à segurança das operações.

O objetivo geral é consolidar a manipulação de arquivos externos com gerenciamento seguro de recursos, tratamento de exceções de entrada e saída (E/S) e preservação da integridade dos dados. Os objetivos específicos são:

- ler arquivos CSV e JSON e converter dados entre os dois formatos sem perda de registros ou de valores;
- utilizar o gerenciador de contexto `with open(...)` com codificação UTF-8 em todas as aberturas de arquivo;
- tratar `FileNotFoundError`, `PermissionError`, erros de codificação e de formato, exibindo mensagens claras sem encerrar o programa de forma inesperada;
- criar uma cópia de segurança automática antes de qualquer operação que altere um arquivo persistido;
- validar as funcionalidades por meio de testes automatizados e de execuções reais, registradas em capturas de tela.

3 Descrição da solução

3.1 Organização do projeto

A solução foi organizada de forma modular, separando a interface com o usuário, a lógica de persistência e o mecanismo de backup. A estrutura do repositório é a seguinte:

```
gerenciador-csv-json/
├── src/
│   ├── main.py        # menu de terminal (interface)
│   ├── arquivos.py    # leitura, escrita, conversão, atualização e exceções
│   └── backup.py      # criação da cópia de segurança (.bak)
├── tests/
│   └── test_arquivos.py   # testes automatizados (unittest)
├── dados/             # produtos.csv, produtos.json, produtos_convertidos.csv
├── backups/           # cópias geradas automaticamente
├── evidencias/        # capturas de tela
└── README.md
```

Módulo	Responsabilidade
`main.py`	Exibe o menu, recebe as entradas do usuário, chama as funções do módulo de persistência e apresenta as mensagens de resultado ou de erro.
`arquivos.py`	Lê CSV e JSON, converte entre os formatos, atualiza registros, grava arquivos e converte falhas de E/S em uma exceção controlada (`ErroPersistencia`).
`backup.py`	Copia o arquivo existente para a pasta `backups` antes de ele ser sobrescrito.
`test_arquivos.py`	Sete testes automatizados com `unittest`, executados sobre arquivos temporários.

3.2 Funcionalidades e persistência dos dados

O menu oferece seis opções: ler e exibir o CSV; converter CSV para JSON; converter JSON para CSV; atualizar um campo de produto no JSON; ler e exibir o JSON; e sair. Os dados de exemplo representam um cadastro de produtos com os campos `id`, `nome`, `categoria`, `preco` e `quantidade`, armazenados em `dados/produtos.csv` e `dados/produtos.json`.

No CSV, cada linha é um registro e a primeira linha contém o cabeçalho. Na leitura, cada registro vira um dicionário Python; na gravação em JSON, a lista de dicionários é serializada como uma lista de objetos. Como o módulo `csv` trata todos os valores como texto, a conversão para JSON preserva os valores como *strings* (por exemplo, `"25.90"`), o que garante que nenhum valor seja alterado ou arredondado durante a conversão. Toda operação que grava um arquivo já existente é precedida por um backup.

4 Implementação

4.1 Gerenciador de contexto e codificação

Todas as aberturas de arquivo utilizam `with`, o que garante o fechamento automático do arquivo ao fim do bloco, inclusive quando ocorre uma exceção. A codificação é sempre declarada: `utf-8` para JSON e para a escrita de CSV, e `utf-8-sig` na leitura de CSV, que também aceita arquivos salvos com marca BOM (comum em planilhas). O parâmetro `newline=""` segue a recomendação da documentação do módulo `csv`. A função abaixo, de `arquivos.py`, ilustra o uso de `with`, a leitura do CSV e o tratamento de exceções (seção 4.4):

```
def ler_csv(caminho: str | Path) -> list[dict[str, str]]:
    """Lê um CSV UTF-8 e retorna uma lista de dicionários."""
    caminho = Path(caminho)
    try:
        with caminho.open("r", encoding="utf-8-sig", newline="") as arquivo:
            leitor = csv.DictReader(arquivo)
            if leitor.fieldnames is None:
                raise ErroPersistencia("O CSV está vazio ou não possui cabeçalho.")
            return [dict(registro) for registro in leitor]
    except FileNotFoundError as erro:
        raise ErroPersistencia(f"Arquivo CSV não encontrado: {caminho}") from erro
    except PermissionError as erro:
        raise ErroPersistencia(f"Sem permissão para ler: {caminho}") from erro
    except UnicodeDecodeError as erro:
        raise ErroPersistencia(
            f"Não foi possível decodificar {caminho} como UTF-8."
        ) from erro
    except csv.Error as erro:
        raise ErroPersistencia(f"Erro ao interpretar o CSV: {erro}") from erro
```

4.2 Leitura de JSON

A leitura de JSON valida a estrutura esperada: o arquivo deve conter uma lista de objetos. Qualquer outro formato é rejeitado com mensagem explicativa. Erros de sintaxe do JSON informam a linha e a coluna da falha.

```
def ler_json(caminho: str | Path) -> list[dict[str, Any]]:
    """Lê um JSON cuja estrutura principal é uma lista de objetos."""
    caminho = Path(caminho)
    try:
        with caminho.open("r", encoding="utf-8") as arquivo:
            dados = json.load(arquivo)
        if not isinstance(dados, list) or any(not isinstance(item, dict) for item in dados):
            raise ErroPersistencia("O JSON deve conter uma lista de objetos.")
        return dados
    except FileNotFoundError as erro:
        raise ErroPersistencia(f"Arquivo JSON não encontrado: {caminho}") from erro
    # ... PermissionError e UnicodeDecodeError tratados como em ler_csv ...
    except json.JSONDecodeError as erro:
        raise ErroPersistencia(
            f"JSON inválido (linha {erro.lineno}, coluna {erro.colno})."
        ) from erro
```

4.3 Escrita e conversão entre CSV e JSON

A gravação é feita por funções internas que, antes de abrir o destino para escrita, chamam `criar_backup`. Na gravação do JSON é usado `ensure_ascii=False`, para que acentos sejam gravados como caracteres legíveis, e `indent=4`. Na gravação do CSV, os nomes das colunas são definidos pelo primeiro registro, e o módulo verifica se todos os registros possuem os mesmos campos na mesma ordem, recusando a conversão em caso contrário, para não perder nem deslocar valores.

```
def _gravar_json(caminho: Path, dados: list[dict[str, Any]], pasta_backups: Path) -> Path | None:
    backup = criar_backup(caminho, pasta_backups)
    with caminho.open("w", encoding="utf-8", newline="") as arquivo:
        json.dump(dados, arquivo, ensure_ascii=False, indent=4)
        arquivo.write("\n")
    return backup


def _gravar_csv(caminho: Path, dados: list[dict[str, Any]], pasta_backups: Path) -> Path | None:
    if not dados:
        raise ErroPersistencia("Não é possível criar CSV sem registros para definir as colunas.")
    campos = list(dados[0].keys())
    if any(list(registro.keys()) != campos for registro in dados):
        raise ErroPersistencia(
            "Os registros do JSON possuem campos diferentes ou em ordem diferente.")
    backup = criar_backup(caminho, pasta_backups)
    with caminho.open("w", encoding="utf-8", newline="") as arquivo:
        escritor = csv.DictWriter(arquivo, fieldnames=campos, extrasaction="raise")
        escritor.writeheader()
        escritor.writerows(dados)
    return backup
```

As conversões combinam leitura e gravação. Observa-se que a leitura da origem ocorre primeiro: se ela falhar, nenhum arquivo de destino é tocado.

```
def converter_csv_para_json(origem, destino, pasta_backups) -> Path | None:
    registros = ler_csv(origem)
    return _gravar_json(Path(destino), registros, Path(pasta_backups))


def converter_json_para_csv(origem, destino, pasta_backups) -> Path | None:
    registros = ler_json(origem)
    return _gravar_csv(Path(destino), registros, Path(pasta_backups))
```

A atualização de um campo lê o JSON, localiza o primeiro registro cujo identificador coincide com o valor informado, altera o campo e regrava o arquivo (com backup). Se o registro ou o campo não existirem, uma exceção controlada é levantada antes de qualquer gravação:

```
def atualizar_registro(caminho_json, identificador, valor_identificador,
                       campo, novo_valor, pasta_backups) -> Path | None:
    caminho = Path(caminho_json)
    registros = ler_json(caminho)
    encontrado = False

    for registro in registros:
        if str(registro.get(identificador)) == valor_identificador:
            if campo not in registro:
                raise ErroPersistencia(f"O campo '{campo}' não existe no registro.")
            registro[campo] = novo_valor
            encontrado = True
            break

    if not encontrado:
        raise ErroPersistencia(
            f"Nenhum registro encontrado com {identificador} = '{valor_identificador}'."
        )
```

```
    return _gravar_json(caminho, registros, Path(pasta_backups))
```

4.4 Tratamento de exceções

O tratamento ocorre em duas camadas. Na camada de persistência (`arquivos.py`), as exceções nativas (`FileNotFoundError`, `PermissionError`, `UnicodeDecodeError`, `csv.Error` e `json.JSONDecodeError`) são convertidas em `ErroPersistencia`, com mensagem em português que indica o arquivo e o problema; a instrução `from erro` preserva a causa original. Na camada de interface (`main.py`), o laço do menu captura `ErroPersistencia`, `PermissionError` (por exemplo, durante a criação do backup ou a gravação) e `OSError`, exibe a mensagem e retorna ao menu, evitando o encerramento inesperado.

```
RAIZ = Path(__file__).resolve().parent.parent
DADOS = RAIZ / "dados"
BACKUPS = RAIZ / "backups"
...
        try:
            if opcao == "1":
                registros = ler_csv(DADOS / "produtos.csv")
                for registro in registros:
                    print(registro)
                print(f"Total de registros: {len(registros)}")
            elif opcao == "2":
                backup = converter_csv_para_json(
                    DADOS / "produtos.csv", DADOS / "produtos.json", BACKUPS
                )
                print("Conversão CSV → JSON concluída.")
                print(f"Backup criado: {backup}" if backup
                      else "Destino novo; não havia arquivo anterior para copiar.")
            ...
        except ErroPersistencia as erro:
            print(f"Não foi possível concluir a operação: {erro}")
        except PermissionError:
            print("Não foi possível acessar ou gravar o arquivo: permissão negada.")
        except OSError as erro:
            print(f"Erro de entrada/saída: {erro}")
```

4.5 Backup automático

O backup é implementado no módulo `backup.py`. A função `criar_backup` verifica se o arquivo original existe; se não existir (destino novo), não há o que preservar e a função retorna `None`. Caso exista, cria a pasta `backups` (se necessário) e copia o conteúdo em modo binário (`rb`/`wb`), preservando exatamente os bytes originais. O nome do backup combina o nome do arquivo, data, hora e microssegundos e a extensão `.bak`, o que evita sobrescrever backups anteriores. O backup é chamado **antes** da abertura do destino para escrita e, se falhar, a exceção se propaga e a alteração do original não chega a ser feita. O caminho do backup é retornado e exibido ao usuário pelo menu.

```
def criar_backup(caminho: str | Path, pasta_backups: str | Path) -> Path | None:
    origem = Path(caminho)
    destino_dir = Path(pasta_backups)

    if not origem.exists():
        return None

    destino_dir.mkdir(parents=True, exist_ok=True)
    instante = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
    destino = destino_dir / f"{origem.stem}_{instante}{origem.suffix}.bak"
```

```
    # O conteúdo é copiado em modo binário para preservar os bytes originais.
    with origem.open("rb") as arquivo_origem:
        conteudo = arquivo_origem.read()

    with destino.open("wb") as arquivo_backup:
        arquivo_backup.write(conteudo)

    return destino
```

5 Testes e evidências de execução

Os testes foram feitos em duas frentes: testes automatizados (`unittest`) e execuções reais do menu no terminal do Visual Studio Code, no Windows (PowerShell), com ambiente virtual ativado.

5.1 Testes automatizados

O arquivo `tests/test_arquivos.py` contém sete testes, executados sobre uma pasta temporária criada em `setUp` e removida em `tearDown`, de modo que os arquivos reais do projeto não são afetados. Os testes cobrem leitura de CSV com acentos, conversão CSV→JSON, conversão JSON→CSV, backup antes de sobrescrever o destino, arquivo inexistente, atualização com backup e JSON inválido. Dois deles são mostrados a seguir:

```
def test_backup_antes_de_sobrescrever_destino(self):
    with self.json.open("w", encoding="utf-8") as arquivo:
        arquivo.write('[{"versao": "antiga"}]')
    converter_csv_para_json(self.csv, self.json, self.backups)
    backups = list(self.backups.glob("*.bak"))
    self.assertEqual(len(backups), 1)
    with backups[0].open("r", encoding="utf-8") as arquivo:
        conteudo_backup = arquivo.read()
    with self.json.open("r", encoding="utf-8") as arquivo:
        conteudo_atual = arquivo.read()
    self.assertIn("antiga", conteudo_backup)
    self.assertNotIn("antiga", conteudo_atual)

def test_arquivo_inexistente(self):
    with self.assertRaises(ErroPersistencia):
        ler_csv(self.dados / "nao_existe.csv")
```

Comando de execução: `python -m unittest discover -s tests -v`.

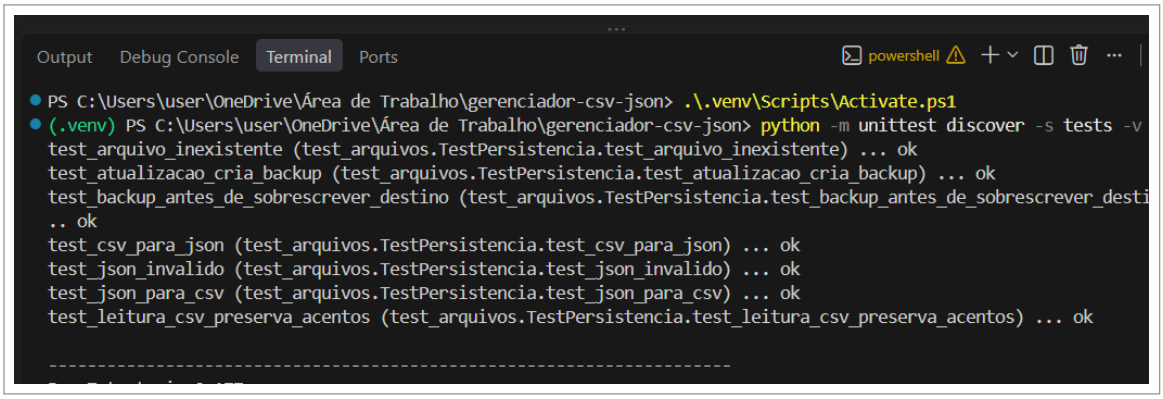


Figura 1 – Execução dos testes automatizados (lista dos testes).

Fonte: elaborado pelo(s) autor(es), 2026.

Cada teste é listado com o resultado `ok`: arquivo inexistente, atualização com backup, backup antes de sobrescrever o destino, CSV→JSON, JSON inválido, JSON→CSV e leitura de CSV com preservação de acentos.

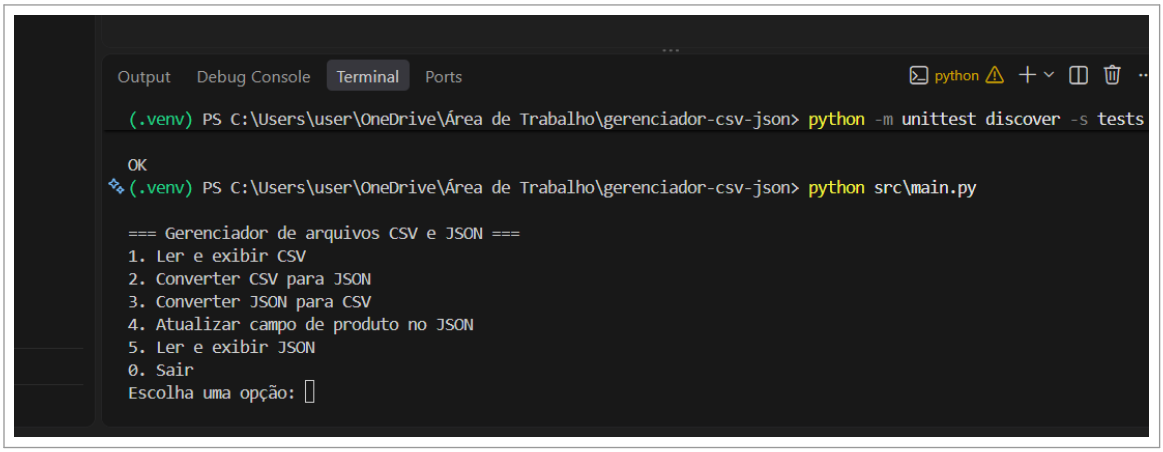


Figura 2 – Resultado final dos testes e abertura do menu.
Fonte: elaborado pelo(s) autor(es), 2026.

A execução termina com `OK`, indicando que os testes passaram. Na sequência, o programa é iniciado com `python src\main.py` e exibe o menu de opções.

5.2 Leitura de CSV e conversão para JSON

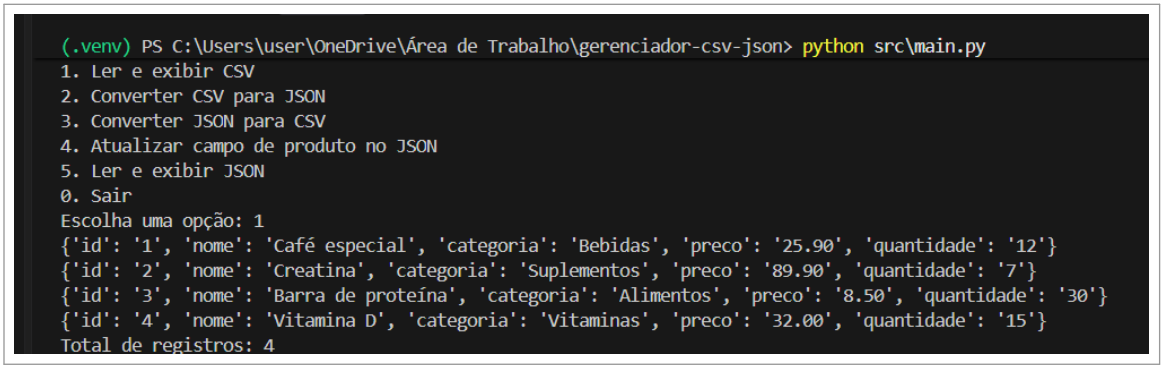


Figura 3 – Opção 1 – leitura e exibição do arquivo CSV.
Fonte: elaborado pelo(s) autor(es), 2026.

O programa lê `dados/produtos.csv` e exibe quatro registros, com acentos preservados ("Café especial", "Barra de proteína"), seguidos do total de registros. Os valores aparecem como texto, comportamento esperado do módulo `csv`.

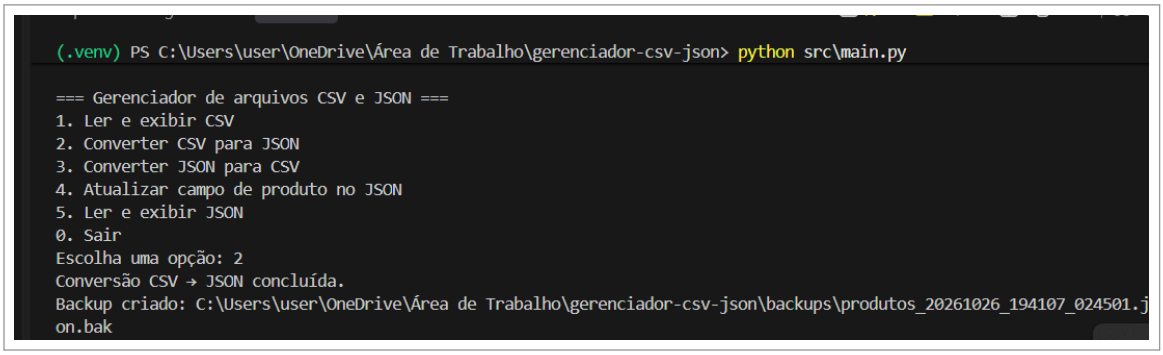


Figura 4 – Opção 2 – conversão de CSV para JSON com backup do destino.

Fonte: elaborado pelo(s) autor(es), 2026.

A conversão é concluída e o programa informa o backup criado em `backups/produtos_20261026_194107_024501.json.bak`. Como o arquivo `produtos.json` já existia, a versão anterior foi copiada antes de ser sobrescrita.

5.3 Leitura de JSON e conversão para CSV

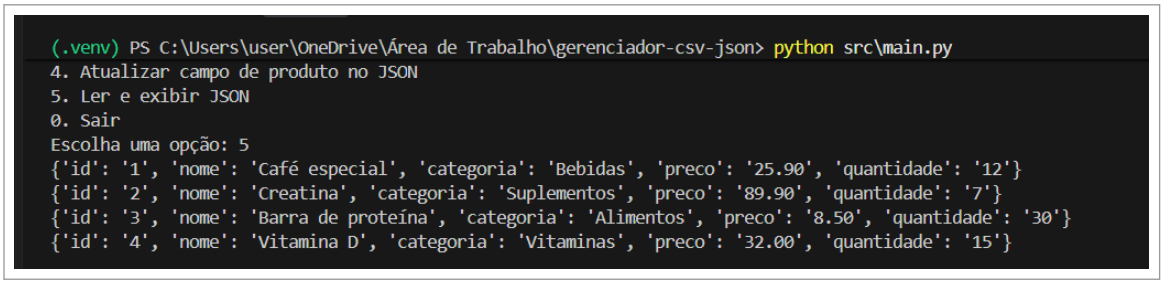


Figura 5 – Opção 5 – leitura e exibição do arquivo JSON.

Fonte: elaborado pelo(s) autor(es), 2026.

O JSON gerado na conversão é lido e apresenta os mesmos quatro registros do CSV, com os mesmos valores, o que demonstra que a conversão não perdeu informações.

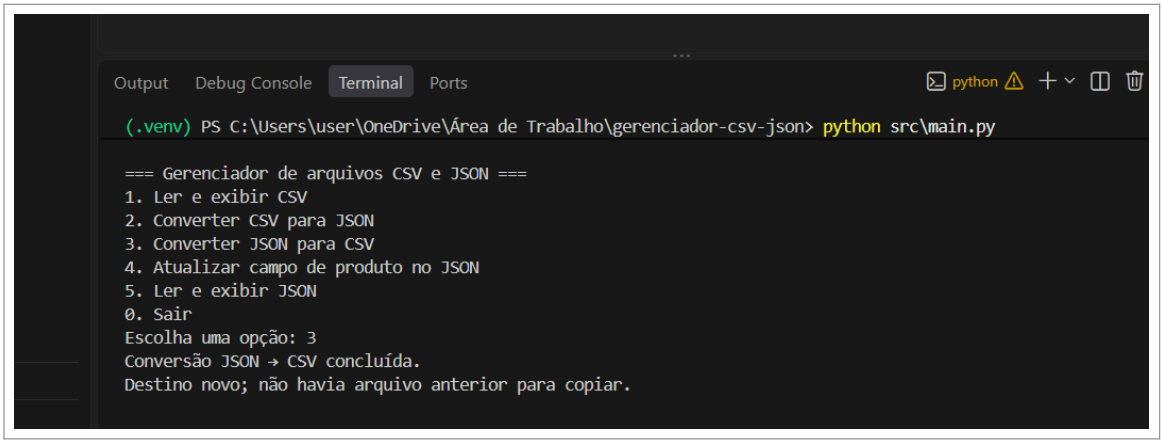


Figura 6 – Opção 3 – conversão de JSON para CSV.

Fonte: elaborado pelo(s) autor(es), 2026.

A conversão é concluída e grava `dados/produtos_convertidos.csv`. Como o destino ainda não existia, o programa informa "Destino novo; não havia arquivo anterior para copiar", e nenhum

backup é criado nesse caso, pois não há dado anterior a preservar.

5.4 Atualização com geração prévia de backup

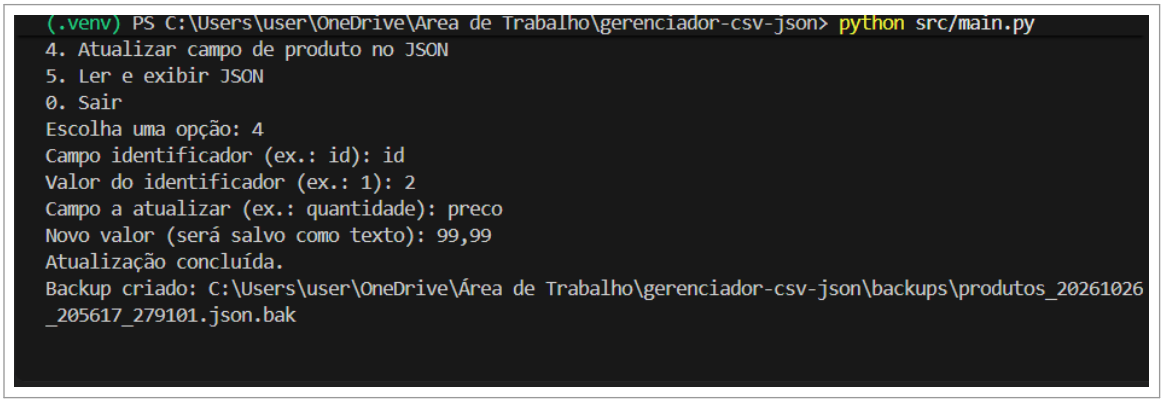


Figura 7 – Opção 4 – atualização do campo *preco* do produto de *id* 2.
Fonte: elaborado pelo(s) autor(es), 2026.

O usuário informa o campo identificador (`id`), o valor do identificador (`2`), o campo a atualizar (`preco`) e o novo valor (`99,99`). O programa confirma a atualização e informa o backup criado, `produtos_20261026_205617_279101.json.bak`.

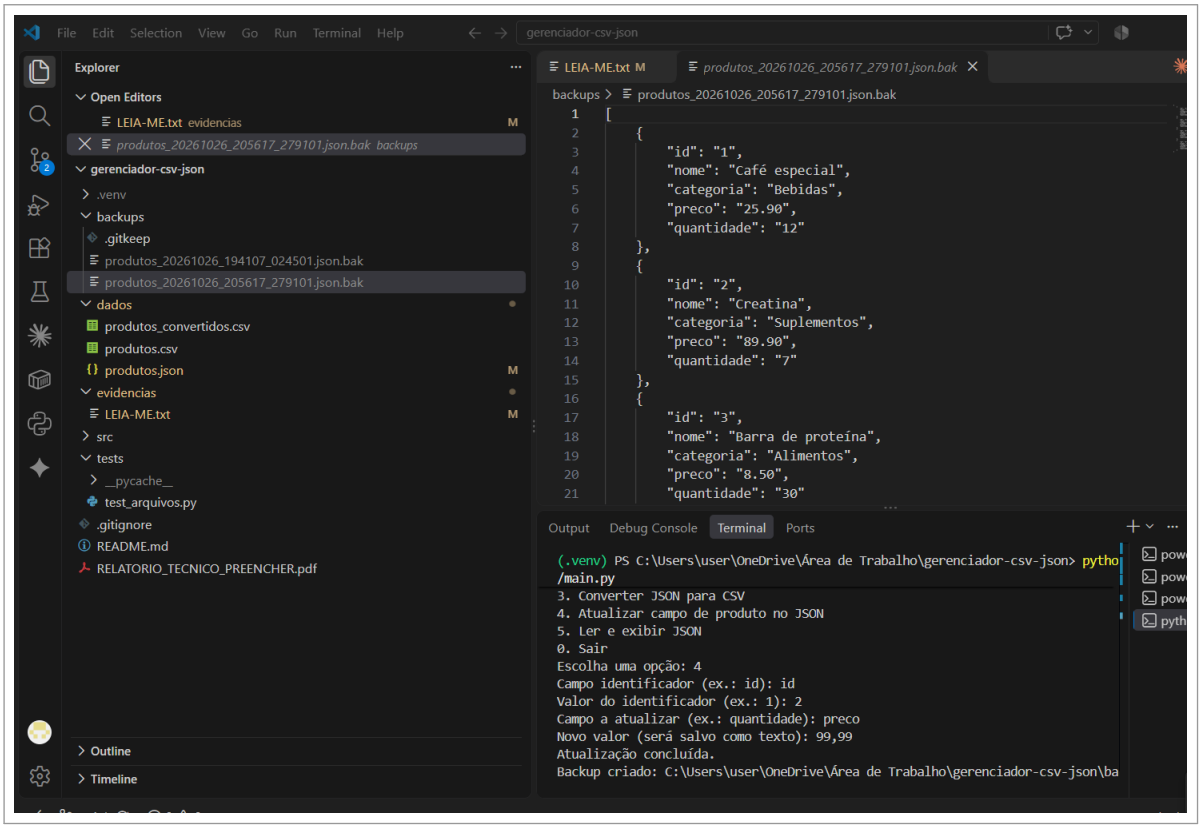


Figura 8 – Pasta *backups* e conteúdo do arquivo de backup gerado.
Fonte: elaborado pelo(s) autor(es), 2026.

No explorador de arquivos aparecem os dois backups criados durante os testes. O arquivo `produtos_20261026_205617_279101.json.bak`, aberto no editor, guarda o estado anterior à

atualização: o produto "Creatina" ainda aparece com `"preco": "89.90"`. No terminal, abaixo, está o registro da operação que o gerou. Comparando com a Figura 5, que mostra o JSON antes da atualização (Creatina com 89.90), nota-se que o backup guardou o valor original.

5.5 Tratamento de erro de arquivo

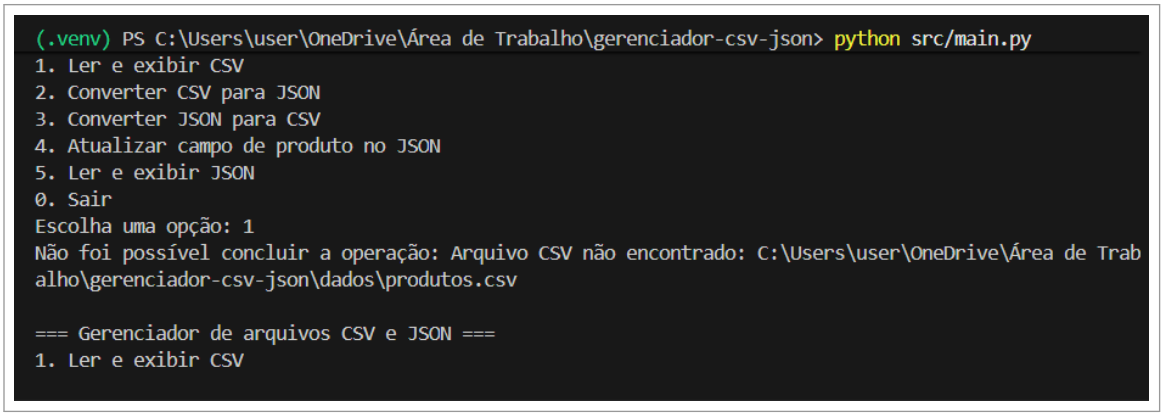


Figura 9 – Opção 1 com o arquivo *produtos.csv* ausente.
Fonte: elaborado pelo(s) autor(es), 2026.

Para provocar o erro, o arquivo `dados/produtos.csv` foi temporariamente renomeado para `produtos_off.csv`. Ao escolher a opção 1, o programa exibiu a mensagem "Não foi possível concluir a operação: Arquivo CSV não encontrado", seguida do caminho do arquivo, e retornou ao menu sem encerrar de forma inesperada. Em seguida o arquivo recebeu de volta o nome original.

Além desse cenário, os testes automatizados verificam o tratamento de JSON inválido (`test_json_invalido`) e de arquivo inexistente (`test_arquivo_inexistente`).

6 Conclusão

O projeto atendeu aos objetivos propostos. Foram implementadas a leitura de CSV e JSON, as conversões nos dois sentidos, a atualização de registros e a gravação com UTF-8, sempre com o gerenciador de contexto `with`. As evidências mostram que os quatro registros de exemplo foram mantidos, com acentos e valores intactos, ao longo das conversões.

O mecanismo de backup automático funcionou conforme o previsto: as capturas mostram a cópia sendo criada antes da sobrescrita e o conteúdo anterior preservado no arquivo `.bak`. O tratamento de exceções, em duas camadas, transformou falhas de E/S em mensagens claras, e o programa permaneceu em funcionamento depois do erro de arquivo ausente. Os sete testes automatizados passaram.

Alguns pontos exigiram atenção no desenvolvimento e merecem registro. Como o CSV não guarda tipos, os valores são tratados como texto, o que evita perdas, mas significa que números não são convertidos automaticamente. Na atualização, o novo valor também é salvo como texto e não passa por validação de formato: na execução registrada, o valor `99,99` foi aceito com vírgula decimal, diferentemente dos demais preços, que usam ponto. Uma melhoria

futura seria validar o formato dos campos numéricos. Também seria possível limitar a quantidade de backups mantidos, já que cada operação de gravação cria um novo arquivo na pasta `backups`.

7 Referências

PYTHON SOFTWARE FOUNDATION. **csv — CSV File Reading and Writing**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/csv.html. Acesso em: 26 out. 2026.

PYTHON SOFTWARE FOUNDATION. **json — JSON encoder and decoder**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/json.html. Acesso em: 26 out. 2026.

PYTHON SOFTWARE FOUNDATION. **pathlib — Object-oriented filesystem paths**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/pathlib.html. Acesso em: 26 out. 2026.

PYTHON SOFTWARE FOUNDATION. **unittest — Unit testing framework**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/unittest.html. Acesso em: 26 out. 2026.

PYTHON SOFTWARE FOUNDATION. **Built-in Functions: open()**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/functions.html#open. Acesso em: 26 out. 2026.

PYTHON SOFTWARE FOUNDATION. **Errors and Exceptions**. The Python Tutorial. Disponível em: https://docs.python.org/3/tutorial/errors.html. Acesso em: 26 out. 2026.

COGHLAN, N.; VAN ROSSUM, G. **PEP 343 – The "with" Statement**. Python Enhancement Proposals. Disponível em: https://peps.python.org/pep-0343/. Acesso em: 26 out. 2026.